

# SchemaSense

---

A Fully Local Multi-Agent Vision-Language System  
for Engineering Diagram Understanding

A local-first pipeline that decomposes diagram understanding into tiling, symbol spotting, connector construction, graph reasoning, caching, and visual verification. The system preserves intermediate artifacts so each answer can be inspected rather than treated as a black box.

**Local VLM stack**

Qwen3-VL served through  
llama.cpp

**Multi-agent design**

Cartographer, spotter, connector,  
reasoner

**Inspectable evidence**

Tiles, overlays, graph, timings,  
provenance

**Simple orchestration**

FastAPI, vanilla web UI, NetworkX  
reasoning

**GROUP MEMBERS**

**EL KHAMKHOUMI Naoufal**

**BOUABDILLAH Hachime**

**BAQUA Abdellah**

Implementation, architecture, and evaluation notes

**May 17, 2026**

# Contents

|                                                               |           |
|---------------------------------------------------------------|-----------|
| <b>Executive Summary</b>                                      | <b>3</b>  |
| <b>1 Motivation</b>                                           | <b>3</b>  |
| 1.1 Design Goals . . . . .                                    | 3         |
| <b>2 System Overview</b>                                      | <b>4</b>  |
| <b>3 One Local VLM, Multiple Logical Agents</b>               | <b>4</b>  |
| <b>4 Agent Roles</b>                                          | <b>4</b>  |
| <b>5 Cartographer: Tiling And Coordinates</b>                 | <b>4</b>  |
| <b>6 Symbol Spotter: Visual Detection</b>                     | <b>6</b>  |
| <b>7 Connector: From Symbols To Graphs</b>                    | <b>6</b>  |
| <b>8 Reasoner: Graph-First Answering</b>                      | <b>7</b>  |
| <b>9 Cache And Orchestration Flow</b>                         | <b>8</b>  |
| <b>10 Local Deployment</b>                                    | <b>8</b>  |
| <b>11 User Interface Design</b>                               | <b>8</b>  |
| <b>12 Evaluation Protocol</b>                                 | <b>9</b>  |
| <b>13 Recorded Evaluation Results</b>                         | <b>9</b>  |
| <b>14 Design Discussion</b>                                   | <b>9</b>  |
| 14.1 Why Use Agents If One VLM Can Answer Directly? . . . . . | 10        |
| 14.2 Why Add A Visual Verifier? . . . . .                     | 10        |
| 14.3 Why Avoid Heavy Frameworks? . . . . .                    | 10        |
| <b>15 Limitations</b>                                         | <b>11</b> |
| <b>16 Future Work</b>                                         | <b>11</b> |



# Executive Summary

SCHEMASense is a local-first system for understanding engineering diagrams with a vision-language model (VLM). It is designed around a small group of logical agents rather than a monolithic prompt. The system uses Qwen3-VL served locally through `llama.cpp`, Python modules for orchestration, a FastAPI backend, a vanilla HTML/CSS/JavaScript interface, and NetworkX for explicit graph representation.

The central design idea is that diagram understanding should be inspectable. A direct full-image VLM answer can be useful, but it usually hides the evidence path. SCHEMASense therefore produces intermediate artifacts: tiles, symbol detections, graph nodes, graph edges, reasoning traces, cache state, timings, and final-answer provenance. The current implementation is hybrid: it keeps graph-first reasoning for explainability, while using a full-image visual verifier to reduce failures caused by incomplete intermediate graph construction.

This report focuses on architecture and design decisions. It does not invent or report new benchmark numbers. Numeric evaluation should be regenerated from the current codebase using the included evaluation runner before submission.

## 1 Motivation

Engineering diagrams combine visual symbols, text labels, flow direction, control signals, and domain conventions. A single vision-language model can often answer questions from the whole image, but such answers are difficult to audit. For engineering settings, auditability matters: a user may need to know which component was detected, which line was treated as a connection, and whether the answer came from a graph or from a direct visual reading.

SCHEMASense explores a practical middle ground. It uses one local VLM server for all semantic interpretation, but separates the work into readable pipeline stages. This keeps deployment simple while giving the system a structured evidence trail.

### 1.1 Design Goals

- **Local deployment:** diagram data stays on the machine. No cloud APIs are used.
- **Simple orchestration:** agents are plain Python functions, not external workflow-framework nodes.
- **Visible intermediate artifacts:** detections, graphs, timing, and cache state are returned to the UI.
- **Graph-based reasoning:** detected components become an explicit NetworkX graph.
- **Robust final answers:** a full-image verifier protects against missed detections or sparse graphs.
- **Lightweight interface:** the demo uses FastAPI and vanilla web technologies.

## 2 System Overview

Figure 1 summarizes the end-to-end architecture. The browser sends a diagram and question to FastAPI. The orchestrator coordinates the agents, stores artifacts, and returns a structured response for the interface.

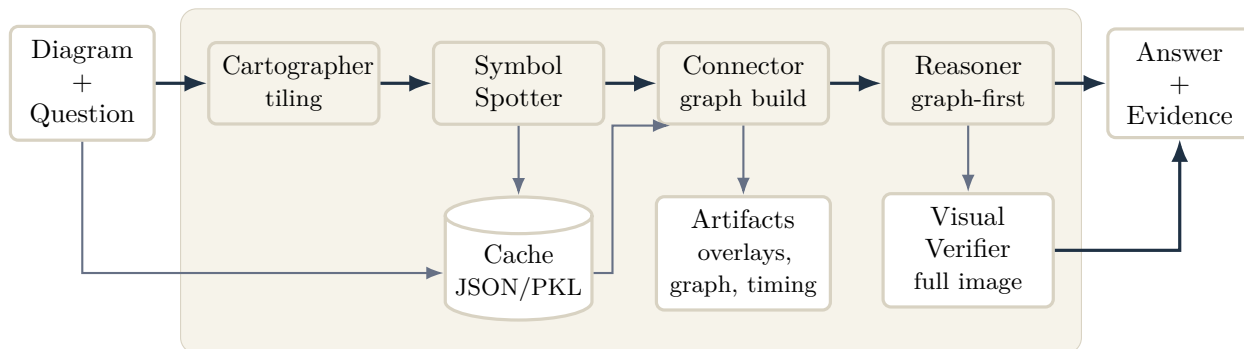


Figure 1: Overall pipeline: the system decomposes diagram understanding into explicit stages while keeping a full-image verifier for final-answer robustness.

## 3 One Local VLM, Multiple Logical Agents

SCHEMASENSE does not load separate models for each agent. Instead, a single local `llama-server` instance exposes an OpenAI-compatible endpoint. Each agent sends a different prompt and, when needed, a different crop or tile.

This design has three advantages:

- **Memory efficiency:** only one VLM instance is needed.
- **Operational clarity:** all model calls pass through one local endpoint.
- **Prompt specialization:** the system can ask the same model to detect, connect, reason, or verify.

## 4 Agent Roles

### 5 Cartographer: Tiling And Coordinates

Large engineering diagrams often contain small tags and symbols. If the entire image is compressed into one model input, small details can be lost. The Cartographer therefore creates overlapping tiles while preserving where each tile came from in the original image.

The important design decision is that tiling is not just preprocessing. It is a coordinate contract. Every downstream detection must remain traceable to the original diagram.

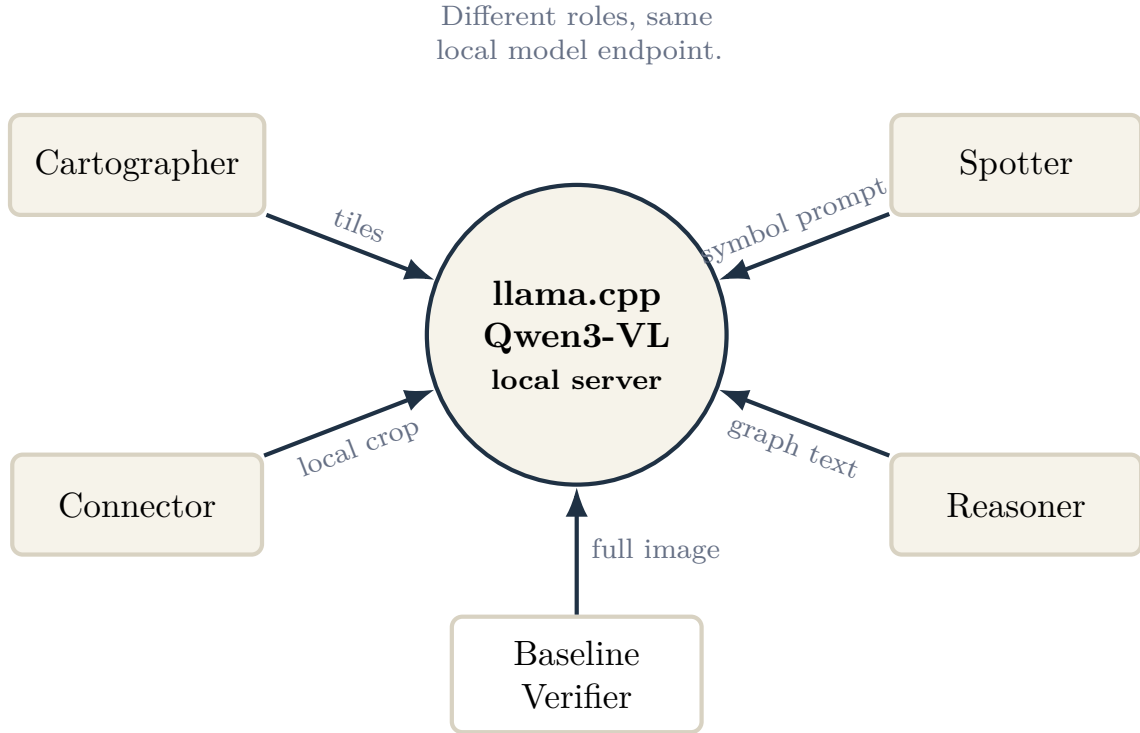


Figure 2: A single local VLM server powers multiple logical agents through specialized prompts and image inputs.

| Agent           | Responsibility                                                                    | Primary Output                                        |
|-----------------|-----------------------------------------------------------------------------------|-------------------------------------------------------|
| Cartographer    | Splits large diagrams into overlapping tiles and preserves coordinate transforms. | Tile metadata and full-image coordinate mapping.      |
| Symbol Spotter  | Detects canonical engineering symbols and normalizes VLM bounding boxes.          | Symbol detections with labels, boxes, and confidence. |
| Connector       | Infers direct connections between detected components.                            | NetworkX graph nodes and edges.                       |
| Reasoner        | Answers from graph evidence and requests visual lookup when needed.               | Graph-grounded answer and reasoning text.             |
| Visual Verifier | Reads the full image to reduce graph error propagation.                           | Robust fallback answer and provenance metadata.       |

Table 1: Agent responsibilities are separated by evidence type rather than by model type.

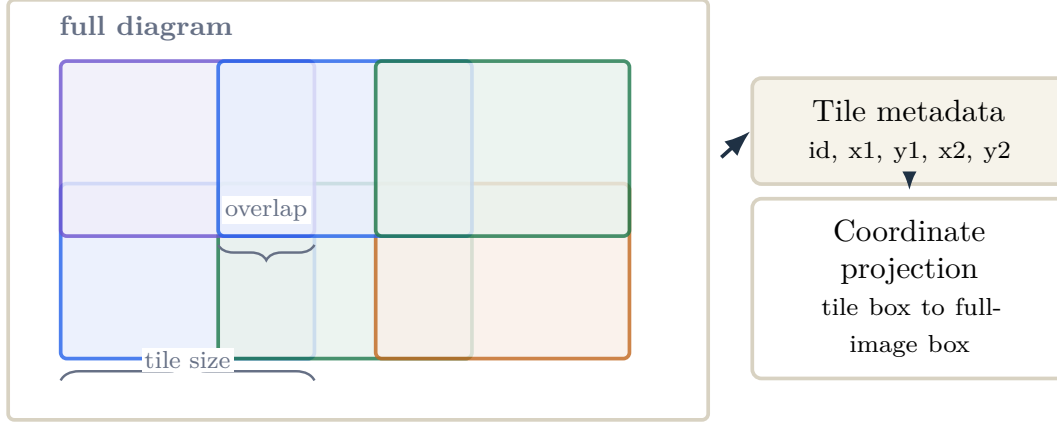


Figure 3: The Cartographer improves local visibility while preserving full-image coordinates for later overlays and graph construction.

## 6 Symbol Spotter: Visual Detection

The Symbol Spotter asks the local VLM to identify visible engineering symbols and return normalized bounding boxes. The system then filters, normalizes, and deduplicates those boxes. For selected shapes, image geometry can refine loose VLM boxes.

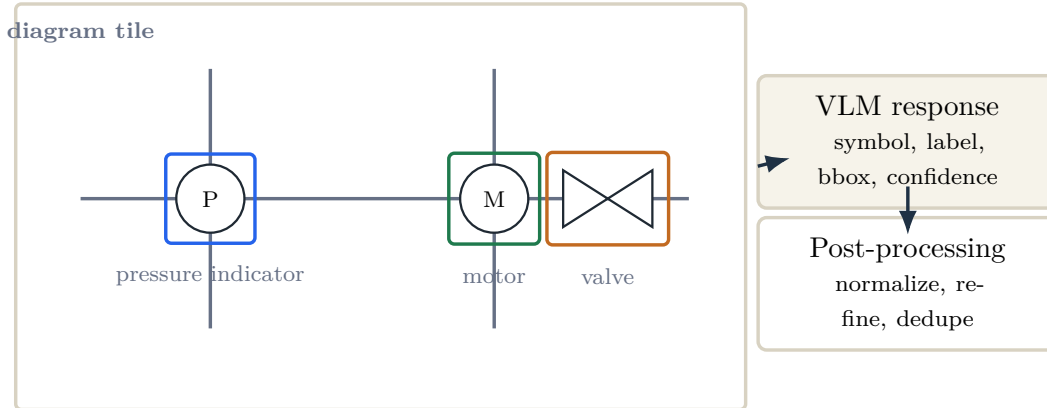


Figure 4: The Spotter turns local visual evidence into structured detections that can be inspected and drawn back onto the source diagram.

The Spotter intentionally uses a controlled vocabulary. This makes outputs easier to normalize, but it also means diagrams dominated by text blocks or non-standard symbols may require full-image verification.

## 7 Connector: From Symbols To Graphs

The Connector converts symbol detections into graph nodes. It then asks localized VLM questions about nearby components: which candidates are directly connected to the target by a pipe, wire, or line?

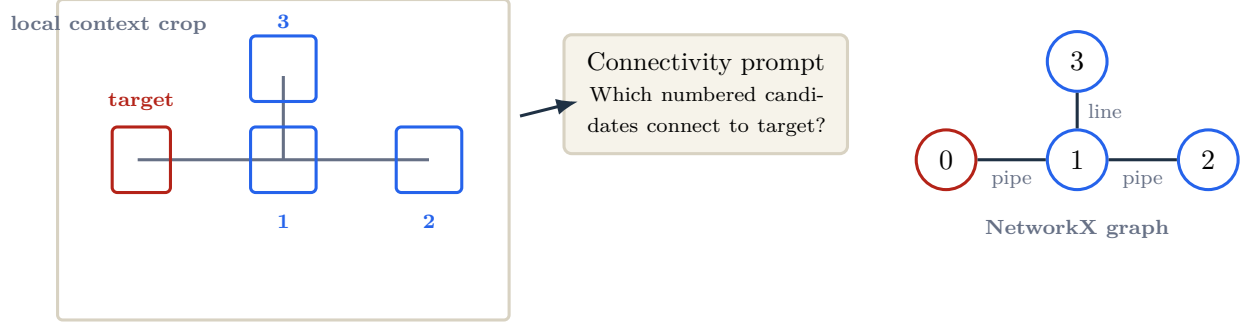


Figure 5: The Connector uses local visual questions to transform detections into an explicit component graph.

This approach is practical for heterogeneous diagrams because it avoids relying entirely on brittle classical line tracing. The tradeoff is that graph quality depends on detection coverage and local context size. For this reason, SCHEMASENSE keeps a full-image verifier in the final-answer path.

## 8 Reasoner: Graph-First Answering

The Reasoner receives the graph as compact text and decides whether the graph is sufficient. If not, it can request visual lookup. In the current system, the orchestrator also runs a full-image visual verifier to protect against graph sparsity.

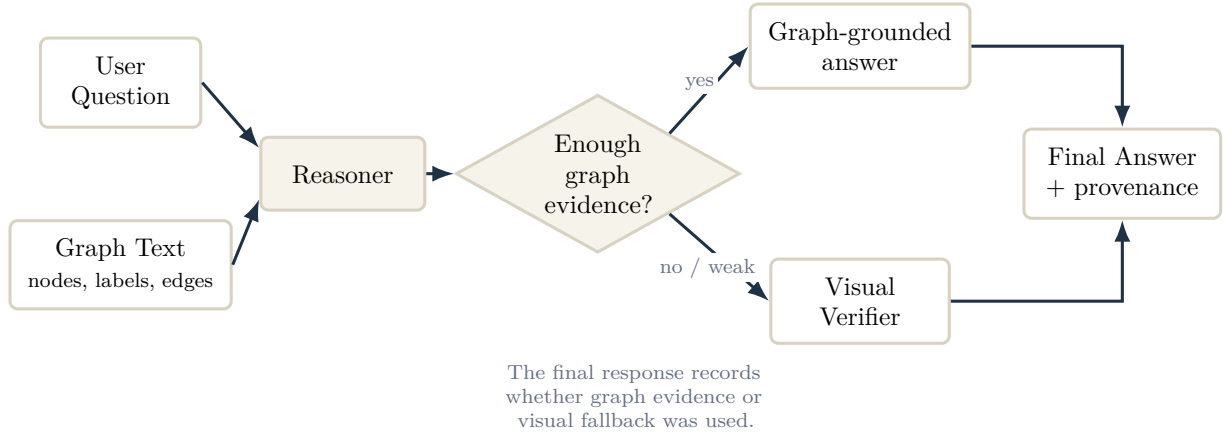


Figure 6: The Reasoner uses graph evidence first, while visual verification improves robustness when graph evidence is incomplete.

The final answer is therefore not just a string. It is accompanied by metadata such as:

- graph answer,
- visual verifier answer,
- confidence,
- timing,

- graph statistics,
- whether image lookup or visual fallback was used.

## 9 Cache And Orchestration Flow

Local VLM calls can be slow, so the orchestrator caches expensive intermediate results. The cache key is derived from the image content, and generated artifacts are written into typed output folders.

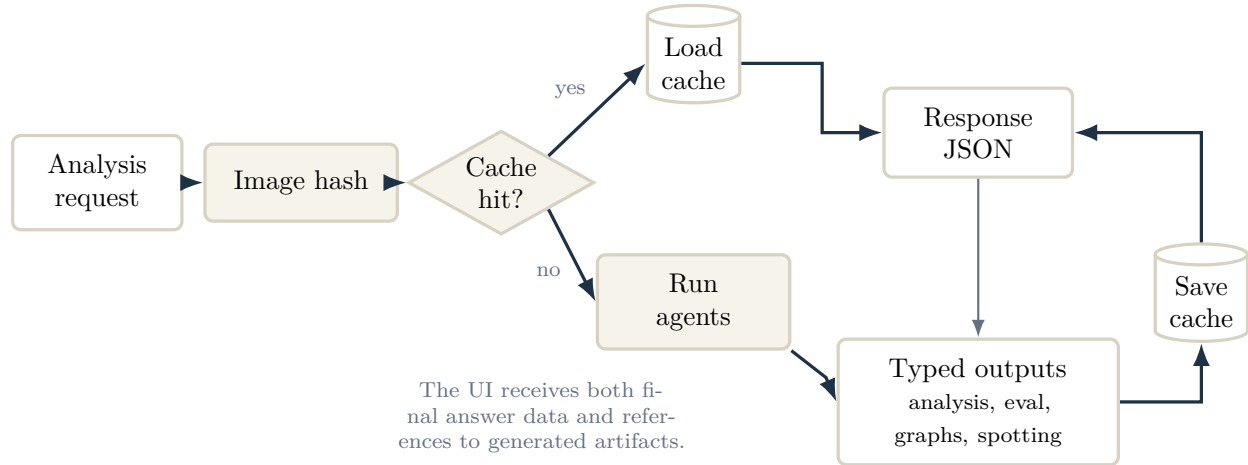


Figure 7: Caching separates repeatable local computation from user-facing response assembly.

## 10 Local Deployment

SCHEMASENSE is designed to run on a single machine:

- `llama-server` hosts Qwen3-VL locally.
- `src/vlm.py` sends OpenAI-compatible requests to the local endpoint.
- FastAPI serves the backend and static frontend.
- The browser UI never needs a cloud service.
- Generated artifacts are written to `outputs/`.

## 11 User Interface Design

The web interface has two views:

- **Analyze:** inspect one diagram-question run, including answer, graph stats, timings, overlays, and trace.
- **Comparison:** launch and stop the full benchmark comparison between the SchemaSense pipeline and the single-shot baseline.

| Layer           | Technology Choice                                                 |
|-----------------|-------------------------------------------------------------------|
| Model serving   | Qwen3-VL through local <code>llama.cpp</code> server              |
| Orchestration   | Pure Python modules and direct function calls                     |
| Backend         | FastAPI endpoints for analysis, evaluation, health, and artifacts |
| Frontend        | Vanilla HTML, CSS, and JavaScript                                 |
| Graph reasoning | NetworkX graph representation                                     |
| Persistence     | JSON artifacts, image overlays, graph files, and local cache      |

Table 2: The stack is intentionally small and local to reduce operational complexity.

The interface is deliberately restrained. It is built for project demonstration and diagnosis rather than marketing. The user can see the current model server, cache state, active stage, progress, and output artifacts.

## 12 Evaluation Protocol

The repository includes `data/questions.json` and `eval.py`. The evaluation compares:

- **SchemaSense:** the multi-agent pipeline with graph reasoning and visual verification.
- **Baseline:** a single full-image VLM prompt with no graph, no tiling, and no intermediate artifacts.

The evaluation runner records predicted answers, correctness labels, timings, and per-type summaries. The current recorded summary in `outputs/results_table.md` is reported below. These values should be regenerated if prompts, routing logic, model files, or grading rules change before final submission.

## 13 Recorded Evaluation Results

The latest available evaluation artifact reports parity between the hybrid SCHEMASENSE pipeline and the single-shot baseline: both systems answered 44 of 45 questions correctly. The remaining error appears in the counting category. Because the current SchemaSense pipeline uses a full-image visual verifier, this result should be interpreted as the performance of the *hybrid multi-agent system*, not as a pure graph-only result.

The result supports the design motivation for the hybrid policy. Graph construction remains useful for transparency and diagnosis, while full-image verification protects the final answer from failures caused by missed symbols, sparse graphs, or text-heavy diagrams.

## 14 Design Discussion

| Method                          | Correct | Total | Accuracy |
|---------------------------------|---------|-------|----------|
| SchemaSense hybrid pipeline     | 44      | 45    | 97.8%    |
| Single-shot full-image baseline | 44      | 45    | 97.8%    |

Table 3: Overall result from `outputs/results_table.md`. The hybrid verifier closes the previous gap to the baseline while preserving intermediate artifacts.

| Question Type  | SchemaSense | Baseline | N  |
|----------------|-------------|----------|----|
| Anomaly        | 4/4         | 4/4      | 4  |
| Connectivity   | 11/11       | 11/11    | 11 |
| Counting       | 14/15       | 14/15    | 15 |
| Identification | 12/12       | 12/12    | 12 |
| Specification  | 3/3         | 3/3      | 3  |

Table 4: Per-type summary from the recorded evaluation table. The only recorded miss is in counting.

### 14.1 Why Use Agents If One VLM Can Answer Directly?

The baseline is useful because it measures raw VLM ability. However, a direct answer does not explain what the system saw or how it reasoned. SCHEMASense adds structure:

- detections show what visual entities were found,
- graph edges show inferred connectivity,
- reasoning traces show whether the answer was graph-grounded or visually verified,
- artifacts make failures easier to diagnose.

### 14.2 Why Add A Visual Verifier?

A graph is only as good as its nodes and edges. If the Spotter misses a text block or the Connector misses a long connection, a pure graph-first system can answer incorrectly. The visual verifier acknowledges this practical limitation. It keeps the interpretability benefits of the graph while preventing the final answer from being trapped by an incomplete intermediate representation.

### 14.3 Why Avoid Heavy Frameworks?

The project is intended as a clear academic demo. Plain Python modules make the pipeline easy to inspect and explain. FastAPI and vanilla frontend code keep deployment lightweight. NetworkX provides enough graph structure without introducing a specialized agent framework.

## 15 Limitations

- The detector vocabulary is still limited for highly heterogeneous diagram families.
- The Connector uses localized visual judgments and can miss long-range relationships.
- The visual verifier improves robustness but reduces the purity of graph-only reasoning.
- Evaluation quality depends on the question set and the grading procedure.
- Large diagrams can require many local VLM calls and may be slow without GPU acceleration.

## 16 Future Work

- Add text and block detection for diagrams dominated by labels and control logic.
- Add directionality, ports, and stream labels to the graph.
- Use cheap geometric line candidates before VLM connectivity verification.
- Add component-level evaluation for detection and graph quality.
- Improve cache invalidation with pipeline-version metadata.
- Produce report figures directly from current run artifacts for reproducible documentation.

## 17 Conclusion

SCHEMASENSE demonstrates a practical local architecture for engineering diagram understanding. Its main contribution is not only answering questions, but making the answer path visible. By combining tiling, symbol detection, graph construction, graph-first reasoning, and full-image verification, the system offers a defensible balance between interpretability and robustness. The design remains simple enough to run locally and explain in a project defense, while leaving clear paths for improving graph quality and evaluation rigor.